EcoLimApp

Digitalización del registro de residuos sólidos

Caso práctico — ECOLIM S.A.C.

Diseño y Desarrollo de Aplicaciones Móviles | PIAD-525

Trabajo Final de Curso — SENATI

El problema actual

Gestión manual de residuos sólidos en ECOLIM S.A.C.

Formatos físicos en papel → traspaso manual a hojas de cálculo

Errores de clasificación

Identificación y clasificación incorrecta de residuos por registro manual.

Retrasos en el registro

Consolidación lenta de la información recolectada en campo.

Reportes inexactos

Datos entregados a las autoridades ambientales con imprecisiones.

Falta de trazabilidad

No hay registro confiable de las operaciones de limpieza realizadas.

La solución

*Una **libreta digital** que reemplaza el papel.*

El operario registra desde su celular qué tipo de residuo recogió, de qué área de la empresa y cuánto pesaba. La app lo guarda ordenado en una base de datos interna y saca las cuentas automáticamente.

Funciona sin internet — modelo offline-first

6 pantallas principales

- **Inicio**
- **Registrar**
- **Historial**
- **Reportes**
- **Estadísticas**
- **Perfil**

Arquitectura del proyecto

Android Studio — organización por capas

3 Activities

LoginActivity (acceso), MainActivity (contenedor) y EscanerActivity (cámara QR/código de barras).

6 Fragments

Inicio, Registrar, Historial, Reportes, Estadísticas y Perfil — se intercambian dentro de Main.

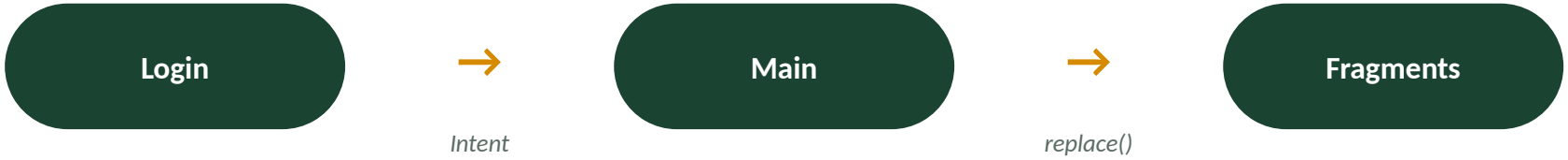
SQLite puro

5 tablas: usuarios, tipos de residuo, áreas, registros y reportes generados. Sin Room.

17 Layouts XML

3 activities, 6 fragments, filas reutilizables, diálogo de reporte y menús.

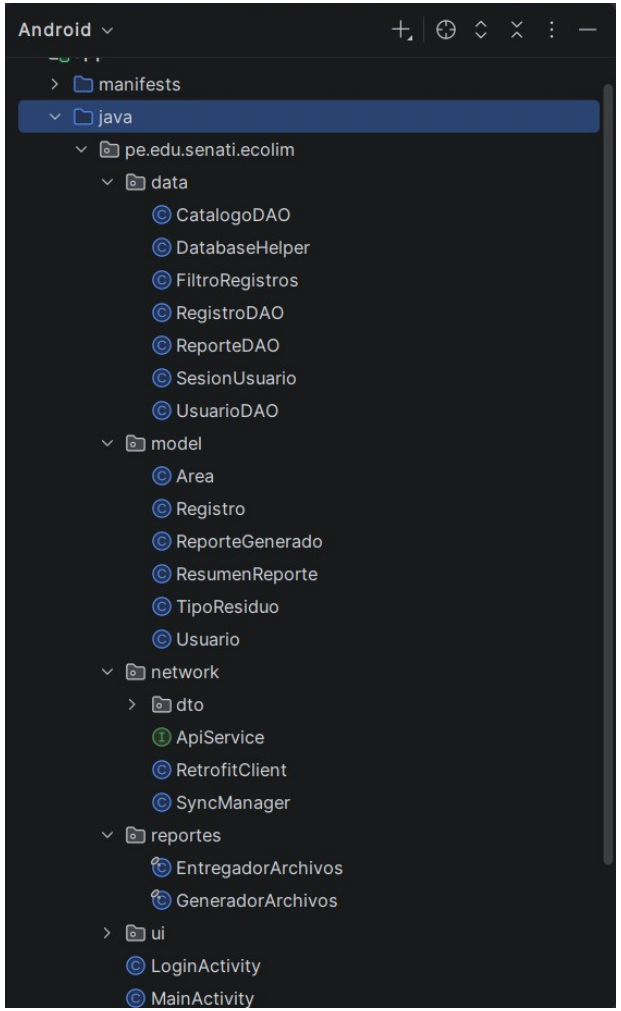
Comunicación entre pantallas



Nota: EscanerActivity se comunica por ActivityResultContracts (sin ViewModel); simple y documentado como oportunidad de mejora.

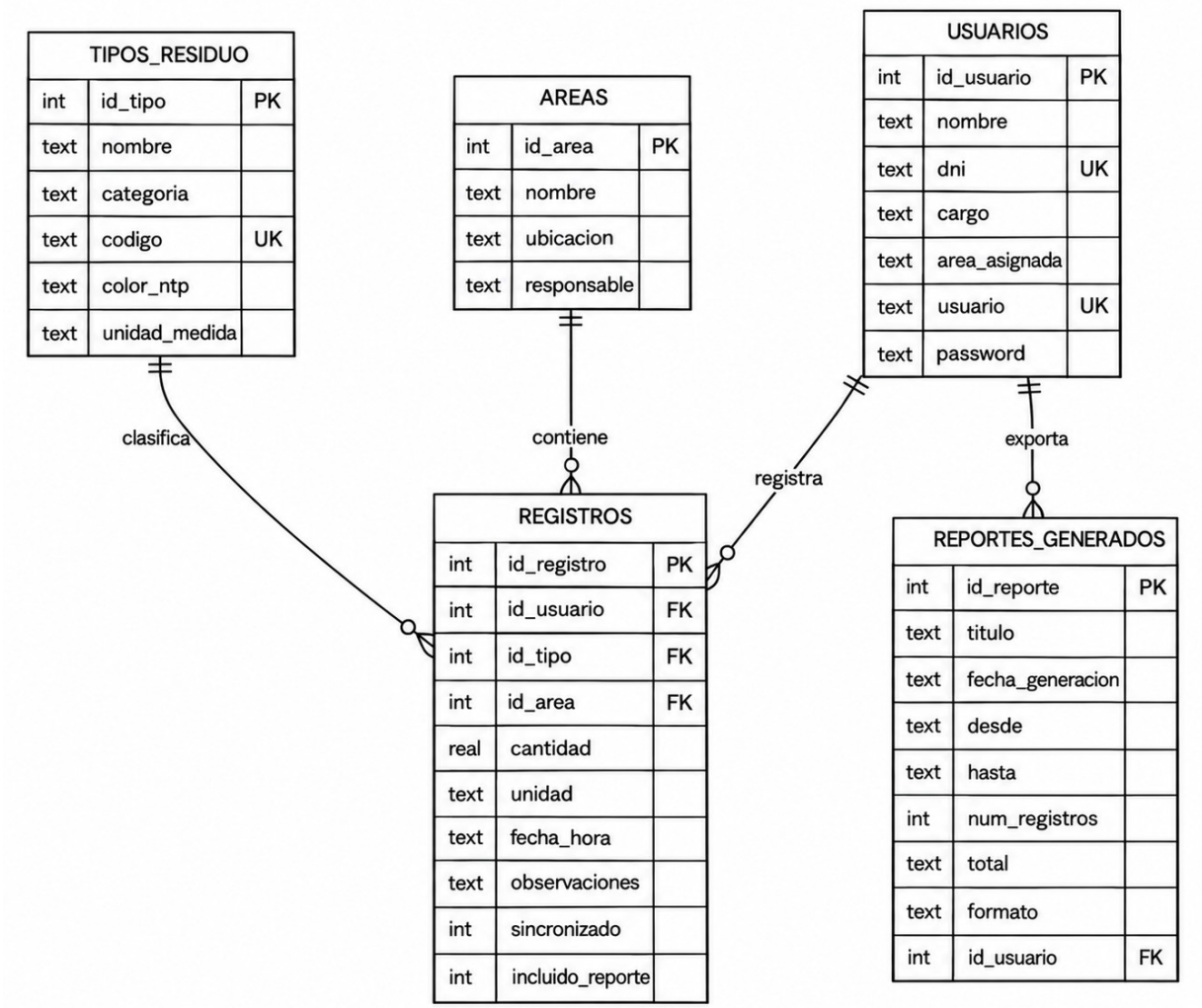
Estructura del proyecto

Android Studio — organización por capas



Modelo de base de datos

5 tablas conectadas mediante llaves foráneas (FOREIGN KEY) — diagrama real del proyecto



Migraciones sin pérdida de datos

onUpgrade() compara versiones (v2→v5) y aplica solo lo nuevo — índices, tabla de reportes, columnas — sin borrar nunca lo existente.

Seguridad de los datos

Contraseñas guardadas como hash SHA-256 (nunca en texto plano) y allowBackup=false, para que la base de datos no salga del dispositivo.

Identificación del tipo de residuo

Doble método — más robusto de lo exigido por el requisito

Menú desplegable (Spinner)

El operario elige el tipo desde una lista cargada de la base de datos — no escribe a mano, imposible equivocarse.

Escaneo de código QR

Cada contenedor tiene un código (ej. RS-PLA). La cámara lo enfoca y la app selecciona el tipo automáticamente.

Al elegir el tipo, la app muestra el color oficial de la norma NTP 900.058:2019 (ej. rojo para peligrosos, verde para vidrio) — conexión directa con la normativa ambiental peruana.

Identificación del tipo de residuo

```
private void onQrEscaneado(String contenido) {
    if (contenido == null || contenido.isEmpty()) {
        return; // el usuario canceló
    }
    String codigo = contenido.trim();
    TipoResiduo t = catalogoDAO.buscarTipoPorCodigo(codigo);
    if (t != null) {
        for (int i = 0; i < tipos.size(); i++) {
            if (tipos.get(i).getIdTipo() == t.getIdTipo()) {
                spinnerTipo.setSelection(i);
                break;
            }
        }
        Toast.makeText(requireContext(), text: "Tipo detectado: " + t.getNombre(), Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(requireContext(),
                text: "Código leído: " + contenido + " (sin coincidencia en el catálogo)",
                Toast.LENGTH_LONG).show();
    }
}
```

Sincronización con la API (offline-first)

El operario registra sin internet; los datos viajan después

1

Guardar localmente

El registro se guarda en el celular con la marca sincronizado = 0 (pendiente).

2

Enviar al servidor

SyncManager busca los registros pendientes y los envía uno por uno vía Retrofit.

3

Confirmar envío

Si el servidor responde OK, se marca sincronizado = 1 (enviado).

Tecnología: Retrofit · Servidor simulado (JSONPlaceholder) · Evidencia en Logcat con la etiqueta "EcoLimSync"

Sincronización con la API(offline – first)

```java
public void sincronizar(final Callback2 cb) {
    final List<Registro> pendientes = registroDAO.pendientes();
    if (pendientes.isEmpty()) {
        cb.onResultado( enviados: 0,  fallidos: 0);
        return;
    }

    final int total = pendientes.size();
    final int[] enviados = {0};
    final int[] fallidos = {0};

    for (Registro r : pendientes) {
        RegistroRequest req = new RegistroRequest(
                r.getIdUsuario(), r.getTipoNombre(), r.getCodigoTipo(), r.getAreaNombre(),
                r.getCantidad(), r.getUnidad(), r.getFechaHora(), r.getObservaciones());

        final long idLocal = r.getIdRegistro();
        Log.d( tag: "EcoLimSync",  msg: "POST registro local=" + idLocal + " codigo=" + r.getCodigoTipo() +
                " tipo=" + r.getTipoNombre() + " cantidad=" + r.getCantidad() + r.getUnidad());
        RetrofitClient.getApi().enviarRegistro(req).enqueue(new Callback<RegistroResponse>() {
            @Override
            public void onResponse(Call<RegistroResponse> call, Response<RegistroResponse> response) {
                if (response.isSuccessful()) {
                    registroDAO.marcarSincronizado(idLocal);
                    enviados[0]++;
                } else {
                    fallidos[0]++;
                }
```

```java
                verificarFin();
            }

            @Override
            public void onFailure(Call<RegistroResponse> call, Throwable t) {
                Log.e( tag: "EcoLimSync",  msg: "Fallo red idLocal=" + idLocal, t);
                fallidos[0]++;
                verificarFin();
            }

            2 usages
            private void verificarFin() {
                if (enviados[0] + fallidos[0] == total) {
                    cb.onResultado(enviados[0], fallidos[0]);
                }
```

Reportes automáticos

Un "filtrador de Excel" dentro del celular

Filtros disponibles

- Texto (tipo, área, observaciones)
- Rango de fechas
- Tipo de residuo
- Unidad (kg / L)
- Cantidad mínima
- Estado (pendiente / sincronizado)

Funciones clave

- Paginación de 10 en 10 registros
- Selección múltiple entre filtros
- Exportación a PDF / CSV / XLSX (compartir o guardar)

Totales y desglose

El resumen ordena el desglose por tipo de residuo, de mayor a menor cantidad.

Decisión técnica: los totales no mezclan kilos con litros — se muestran por separado (ej. "12.5 kg · 3.0 L"), porque sumar peso con volumen sería un error físico.

Historial de reportes generados: cada exportación queda registrada y se puede eliminar con una pulsación larga.

Reportes automáticos(csv)

```java
public static String construirCsv(String titulo, List<Registro> detalle,
                                  List<ResumenReporte> resumen, String totalTexto) {
    StringBuilder csv = new StringBuilder();
    csv.append(titulo).append(" - ECOLIM S.A.C.\n");
    csv.append("Registros incluidos: ").append(detalle.size()).append("\n\n");

    csv.append("DETALLE\n");
    csv.append("Fecha;Tipo;Area;Cantidad;Unidad;Estado\n");
    for (Registro r : detalle) {
        csv.append(r.getFechaHora()).append(";")
                .append(r.getTipoNombre()).append(";")
                .append(r.getAreaNombre()).append(";")
                .append(String.format(Locale.getDefault(), format: "%.1f", r.getCantidad())).append(";")
                .append(r.getUnidad()).append(";")
                .append(r.getSincronizado() == 1 ? "Sincronizado" : "Pendiente").append("\n");
    }

    csv.append("\nRESUMEN POR TIPO\n");
    csv.append("Tipo;Total;N registros\n");
    for (ResumenReporte s : resumen) {
        csv.append(s.getTipoNombre()).append(";")
                .append(String.format(Locale.getDefault(), format: "%.1f", s.getTotalCantidad())).append(";")
                .append(s.getNumRegistros()).append("\n");
    }
    csv.append("\nTotal: ").append(totalTexto).append("\n");
    return csv.toString();
}
```

Publicación en Google Play (simulado)

Permisos declarados

INTERNET + ACCESS_NETWORK_STATE

Para sincronizar con el servidor.

CAMERA (required="false")

Para escanear QR — la app funciona incluso sin cámara.

No pide ubicación — Google Play penaliza permisos que la app no usa realmente.

Seguridad y buenas prácticas

Contraseñas con hash SHA-256 + allowBackup="false"

Los datos nunca salen del dispositivo en texto plano.

Identidad de la app (build.gradle)

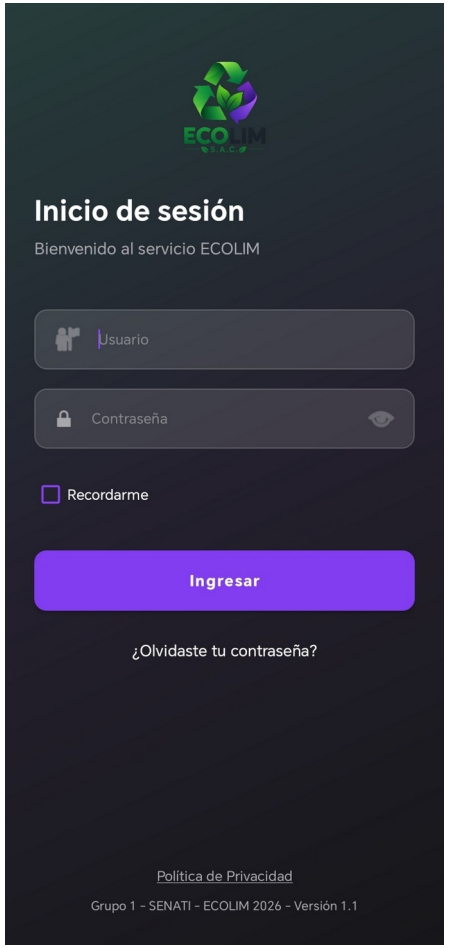
applicationId	pe.edu.senati.ecolim
versionCode	1
versionName	1.0

Un despliegue real exigiría además firmar el APK y quitar el modo debuggable — fuera del alcance del curso.

La versión mostrada en el login se lee de BuildConfig.VERSION_NAME — se actualiza sola en cada build, nunca se escribe a mano.

Evidencias del proyecto

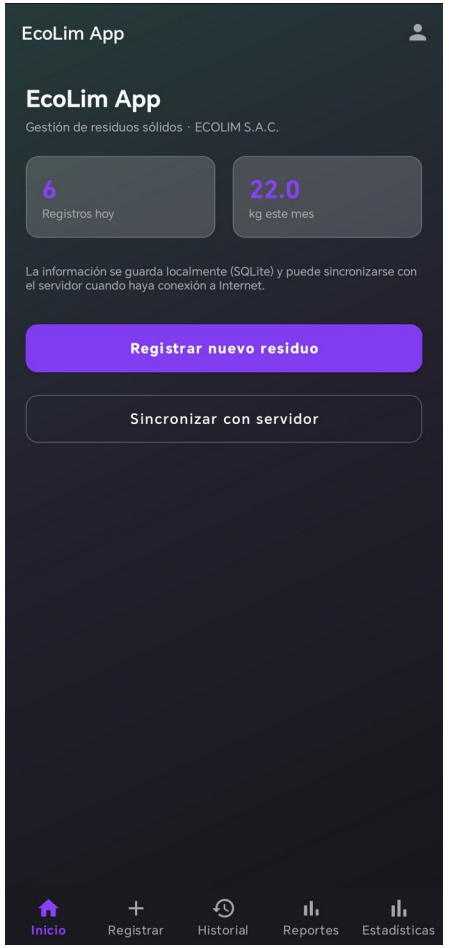
Capturas reales de la app — acceso, perfil, inicio y registro



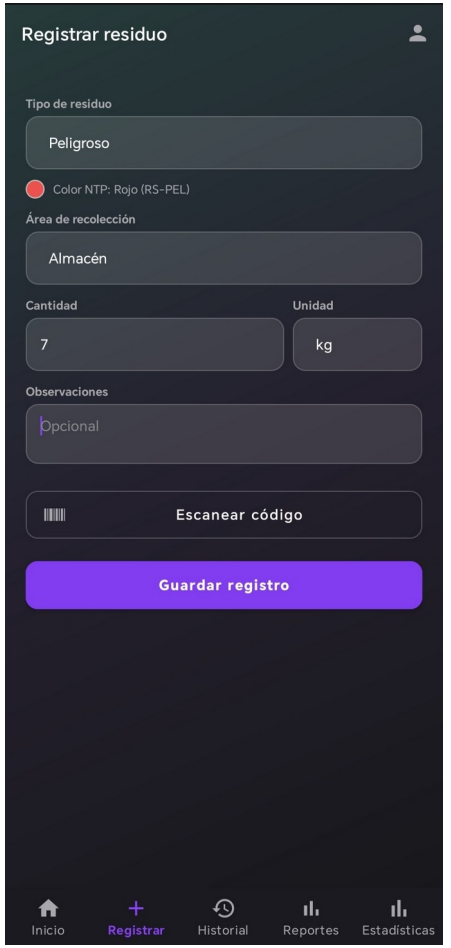
Login



Perfil



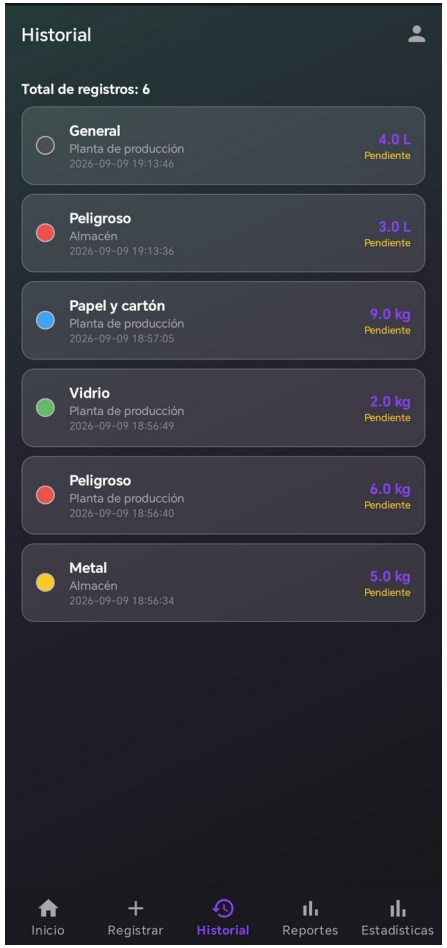
Inicio



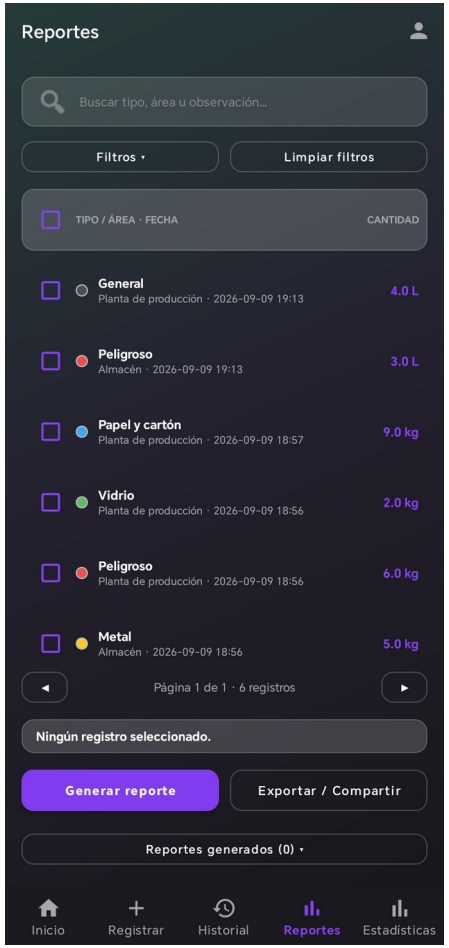
Registro

Evidencias del proyecto

Capturas reales de la app — historial, reportes y estadísticas



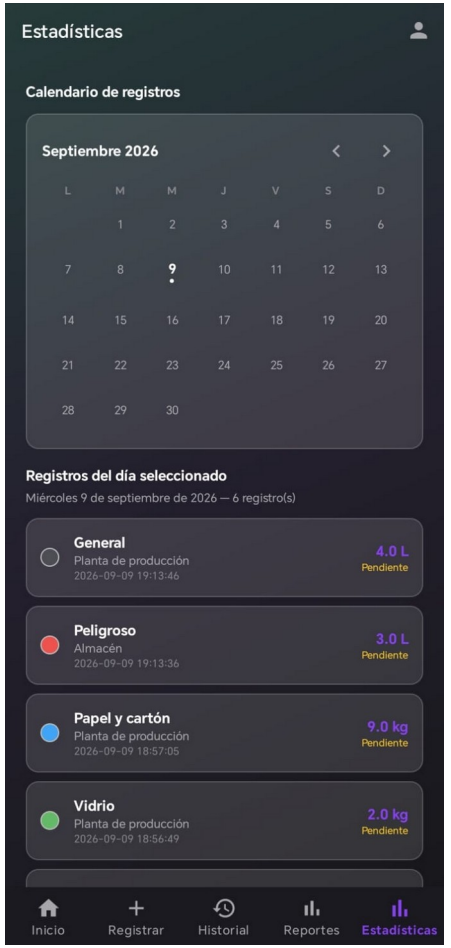
Historial



Reportes



Panel estadístico



Panel estadístico

Conclusiones

Elimina errores de clasificación mediante Spinner + escaneo QR validado contra catálogo.

Resuelve retrasos con registro en tiempo real y sincronización offline-first.

Genera reportes exactos y trazables (PDF/CSV/XLSX), listos para compartir con áreas y autoridades.

Calidad de ingeniería: migraciones sin pérdida de datos, contraseñas hasheadas y exportación sin dependencias pesadas.

Mejora futura identificada: incorporar ViewModel/Repository para compartir estado entre fragments.

¡Gracias!